

Chapter 15 .합성곱 CNN_이미지 증강

- 이미지 증강 API
- cats_dogs 이미지 분류 CNN 모델 구현

* CNN(Convolution Neural Network) 과적합 해결을 위한 이미지 데이터 증강(Data Augmentation)

- CNN 모델의 성능을 향상하기 위해 학습 데이터의 다양성을 늘려야함
- 이미지에 대한 약간의 변형만 있어도 오차가 증가함으로 이미지에 대한 다양성을 확보하여 CNN 모델의 성능을 향상 시켜야함
- 직접 이미지를 생성하는 일은 시간과 비용이 너무 많이 소모
- 원본 이미지를 수평,수직 반전, 회전, cropping(잘라내기), 색조 변경등 다양한 방법으로 변경하여 데이터를 증강 시킴
- 원본 이미지 증강 기법 ==> Tensorflow.keras 의 ImageDataGenerator 가 대신 제공

여러가지 이미지 데이터 증강 형태



rotation_range=90



원본 이미지



zoom_range=0.5



height_shift_range=0.2



vertical_flip=True

1. ImageDataGenerator 클래스

- **from tensorflow.keras.preprocessing.image import ImageDataGenerator**
- 여러 데이터 증강 유형 설정으로 원본 이미지 데이터 증강 준비



원본이미지
데이터



ImageDataGenerator

1. 데이터 증강 유형 설정(rescale, 회전, 등등)

```
train_image_generator=ImageDataGenerator(  
rescale=1.0/255., # 이미지 값을 0과 1사이로 변경  
rotation_range=90, # 무작위 회전각도 90도 이내  
height_shift_range=0.2) # 상하 이동 비율
```

2. x,y 설정, 배치사이즈, 데이터 셔플 유무 등등 설정 과 동시 Numpy Array Iterator 객체 생성

```
train_data_gen =  
train_image_generator.flow_from_directory(  
train_dir,  
batch_size=2,  
shuffle=False,  
target_size=(150,150) )
```

2. flow() Or flow_from_directory() 메서드 호출로 NumpyArrayIterator 객체 Or DirectoryIterator 객체 생성

flow_from_directory() 메서드

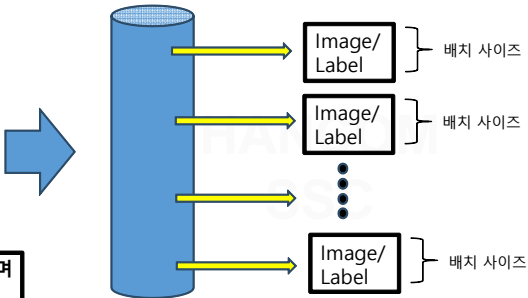
2. x,y 설정, 배치사이즈, 데이터 셔플 유무 등등 설정
과 동시 Numpy Array Iterator 객체 생성

train_data_gen =

```
train_image_generator.flow_from_directory(  
    train_dir,  
    batch_size=2,  
    shuffle=True,  
    class_mode='binary',  
    target_size=(150,150) )
```

- train_dir의 하위 디렉토리 이름을 레이블로 간주하며
해당레이블이라고 간주한 디렉토리 아래의 파일을
해당 레이블의 이미지들 이라고 알아서 추측하여
수행

Iterator



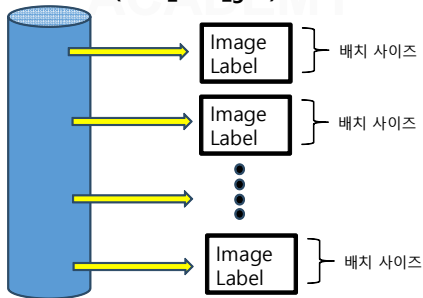
Iterator 반복 순회시
설정된 batch_size 만큼 이미지 데이터 추출

3. 모델 fit() 호출 (증강 이미지 추출 생성 및 훈련)

: flow(), flow_from_directory()로 **생성된 Iterator** 전달

: steps_per_epoch=200, epochs=100, validation_steps=10, callbacks 등
모델 훈련 옵션 설정

Iterator (train_data_gen)



모델 훈련

```
model.fit(  
    train_data_gen,  
    validation_data=test_data_gen,  
    steps_per_epoch=200, epochs=100,  
    validation_steps=10,  
    verbose=1)
```

• 증강(변형) 이미지 저장 방법

```
train_dir = 'D:/CNN_DataSet/cnn_cats_and_dogs_dataset/train'
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
# 1. 사용자의 이미지 증강 유형이 설정된 ImageDataGenerator 객체 생성으로
```

```
# ==> 이미지 증강 유형 설정
```

```
train_image_generator = ImageDataGenerator( rescale= 1.0/255.,  
                                             rotation_range=90,  
                                             height_shift_range = 0.2 )
```

• 증강(변형) 이미지 저장 방법

2. 불러올 이미지 경로 설정 또는

(image array,label array) 설정, 배치 사이즈, 데이터 셔플 유무,

변형이미지 저장 방법등이 설정된 flow(), flow_from_directory() 메서드 호출로

==> Numpy Array Iterator 객체 생성

flow_from_directory()메서드는

train_dir 의 하위 디렉토리 이름을 레이블이라고 간주하고 해당 레이블이라고 간주한

디렉토리 아래의 파일을 해당 레이블의 이미지들이라고 앞서 추측하여 수행

```
train_data_gen = train_image_generator.flow_from_directory(  
    train_dir,          # 불러올 이미지 경로  
    batch_size=2,       # 배치 사이즈  
    ★shuffle=False,    # False일 경우 이미지를 순서대로 불러와 증강  
    save_to_dir='D:/CNN_DataSet/cnn_cats_and_dogs_dataset/temp',  
    save_prefix='gen',  
    save_format='jpg',  
    target_size=(150,150) ) # CNN 모델 입력 사이즈에 맞게 resize
```


• 증강(변형) 이미지 저장 방법

```
# 3. next()함수 또는 generator 반복 수행으로 ==> 증강 이미지 추출  
# sample_train_images, _ = next(train_data_gen) # 배치사이즈 만큼 증강 이미지 생성  
# next() 함수반환값 : ( x_train(이미지데이터), y_train(라벨) )
```

```
# Iterator 객체를 반복 수행하여 증강(변형) 이미지 확인  
# 실제 모델에서는 Iterator객체를 fit() 메서드로 실행하면 배치사이즈 만큼  
# 증강이미지 생성 후 모델의 인풋으로 입력되어 학습 시작
```

```
# model fit() 2 epoch 훈련시 예제  
#예) steps_per_epoch(45) 중 1 steps_per_epoch당  
# batch_size(4)만큼 이미지 꺼내 이미지 증강 시킬 경우  
# 최종 증강 이미지 파일 ==> 45(steps_per_epoch)*4(batch_size) * 2(epoch) => 360개
```

```
i = 0 # <=== 2 batch_size의 train_data_gen(제너레이터)를 3번 반복시 6개 이미지 추출  
for b in train_data_gen:  
    i+=1  
    if i>2:  
        break
```

해당 디렉토리
내부 생성
이미지 확인

```
save_to_dir='D:/CNN_DataSet/cnn_cats_and_dogs_dataset/temp',
```


* cats_dogs 이미지 분류 CNN 모델 구현

<https://www.kaggle.com/competitions/dogs-vs-cats/data> 캐글 데이터셋 참조 활용

데이터셋 전처리 : ImageDataGenerator() 활용 데이터 정규화,가변,증강

train_dir = 'D:/CNN_DataSet/cnn_cats_and_dogs_dataset/train'

test_dir = 'D:/CNN_DataSet/cnn_cats_and_dogs_dataset/test'

from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_image_generator = ImageDataGenerator(rescale= 1.0/255.,
rotation_range=90)

이미지 값 0~1로 스케일 변환
이미지 회전 90도 이내

검증데이터는 스케일 만 조정

test_image_generator = ImageDataGenerator(rescale= 1.0/255.)

* cats_dogs 이미지 분류 CNN 모델 구현

```
# flow_from_directory() 메서드 사용시 데이터셋이 디렉토리로 구별되어 있어야함
# fit() 동작시 ==> 변형 이미지 생성, 저장, 훈련
# steps_per_epoch * batch_size * epochs 만큼 이미지 증강 / 훈련 ★
# 이터레이터 생성

train_data_gen = train_image_generator.flow_from_directory(train_dir,
                                                           batch_size=20,
                                                           class_mode='binary',
                                                           target_size=(150,150)) # ==> 모델 입력 shape 설정

test_data_gen = test_image_generator.flow_from_directory(test_dir,
                                                         batch_size=20,
                                                         class_mode='binary',
                                                         target_size=(150,150))
```

* flow_from_directory() 메서드 class_mode 분류 방식 지정

class_mode	라벨 반환값	모델 출력 활성화 함수	모델 컴파일 손실함수 매칭
categorical	2D one-hot 부호화된 라벨이 반환	softmax	categorical_crossentropy
binary	1D 이진 라벨이 반환	sigmoid	binary_crossentropy
		softmax	sparse_categorical_crossentropy
sparse	1D 정수 라벨이 반환	softmax	sparse_categorical_crossentropy

* flow_from_directory() 메서드 class_mode 분류 라벨 출력

class_mode='categorical' 설정 예)

```
train_generator = train_datagen.flow_from_directory(  
    train_dir, # train directory 참조  
    target_size=(image_size, image_size), # ( 224, 224 ) 크기  
    batch_size = batch_size, # 4  
    class_mode='categorical',  
    shuffle=True  
)
```

```
import numpy as np  
imgs, labels = [], []  
for i in range(10):  
    a, b = train_generator.next() # 4 batch * 10 ==> 총 40개  
    imgs.extend(a)  
    labels.extend(b)  
  
print(np.asarray(imgs))  
print(np.asarray(labels))  
print(len(np.asarray(labels)))
```

* 실습 코드와 별개

class_mode
='categorical' 경우
one-hot 인코딩 상태

[[1. 0.]
 [0. 1.]
 [1. 0.]
 [1. 0.]
 [1. 0.]
 [0. 1.]
 [1. 0.]
 [0. 1.]
 [1. 0.]
 [1. 0.]
 [1. 0.]
 [0. 1.]
 [1. 0.]
 [1. 0.]
 [1. 0.]
 [1. 0.]
 [1. 0.]

* cats_dogs 이미지 분류 CNN 모델 구현

Conv 신경망 설계

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten, Conv2D, MaxPooling2D

model = Sequential()
model.add(Conv2D(16, kernel_size=(3, 3), input_shape=(150, 150, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=2))
model.add(Conv2D(32, (3, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=2))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=2))
model.add(Flatten())
model.add(Dense(512, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.summary()
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

* cats_dogs 이미지 분류 CNN 모델 훈련(학습)

```
# 첫번째 매개변수에 Batch generator 전달하여 학습 진행
# steps_per_epoch ==> 1 epoch 을 진행할 총 스텝 크기
# 총 4000 개 샘플 ==> 200 * 20(batch_size) ==> 4000
# "steps_per_epoch * batch_size <= 훈련 샘플 수 , 에포크수외 별개
# validation_steps는 한 번의 에포크가 끝날 때, 테스트에 사용되는 배치 (batch)의 개수를 지정
# validation_steps(10) * batch_size(20) ==> 총 테스트 샘플 수(200)
# 모델저장, 조기종료 콜백 추가
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping
checkpoint_cb =
ModelCheckpoint('best_cats_dogs_classification_model.h5',save_best_only=True)
early_stopping_cb = EarlyStopping(patience=3, restore_best_weights=True)

history = model.fit(train_data_gen,
                    validation_data=test_data_gen,
                    steps_per_epoch=200, epochs=50, validation_steps=10,
                    verbose=1, callbacks=[checkpoint_cb, early_stopping_cb])
```

* GPU 사용
7 epoch 기준 약 2분간 훈련

* cats_dogs 이미지 분류 CNN 모델 성능 시각화

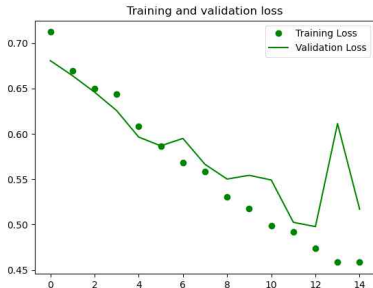
성능 시각화

```
import matplotlib.pyplot as plt
```

```
acc = history.history['accuracy']  
val_acc = history.history['val_accuracy']  
loss = history.history['loss']  
val_loss = history.history['val_loss']
```

```
epochs = range(len(acc))  
plt.figure()
```

```
plt.plot(epochs, loss, 'go', label='Training Loss')  
plt.plot(epochs, val_loss, 'g', label='Validation Loss')  
plt.title('Training and validation loss')  
plt.legend()  
plt.show()
```



* 새로운 임의의 cats_dogs 이미지 분류

```
from tensorflow.keras.models import load_model
```

```
bestcnn_model = load_model('best_cat_dog_clf_model.h5') # 앞의 저장 모델 로딩
```

```
from tensorflow.keras.preprocessing import image # 이미지 전처리 모듈
```

```
# 임의의 예측 데이터 로딩
```

```
img =  
image.load_img('D:/CNN_DataSet/cnn_cats_and_dogs_dataset/predict_img/test_dog.jpg',  
target_size=(150,150))
```



test_cat.jpg



```
img_arr = image.img_to_array(img) / 255.0 # 데이터 정규화  
print(img_arr)  
print(img_arr.shape)
```

* 새로운 임의의 cats_dogs 이미지 분류

새로운 이미지 예측

```
pred = bestcnn_model.predict(img_arr.reshape(1,150,150,3),batch_size=1)  
print(pred)
```

[[0.6409904]]

```
import numpy as np  
pre_result = np.where(pred[0] > 0.5, 1, 0) # cats label : 0, dogs label : 1  
print(pre_result) # [1]
```

```
cat_dog_classnames = np.array(['cat','dog'])
```

```
print(cat_dog_classnames[pre_result])
```

['dog']

Thank You